

基于大模型的软件测试使用记录

目录

1	成员 A 使用记录	2
1.1	测试计划编写	2
1.2	测试代码生成与执行	4
1.3	测试报告编写与修改	5
2	成员 B 使用记录	8
2.1	测试代码生成与执行	8
2.2	测试报告编写与修改	10
3	成员 C 使用记录	12
3.1	Web 端测试代码生成与执行	12
3.2	移动端测试计划与环境确认	14
3.3	移动端测试错误定位与修复	16
3.4	测试报告编写与修改	19

1 成员 A 使用记录

1.1 测试计划编写

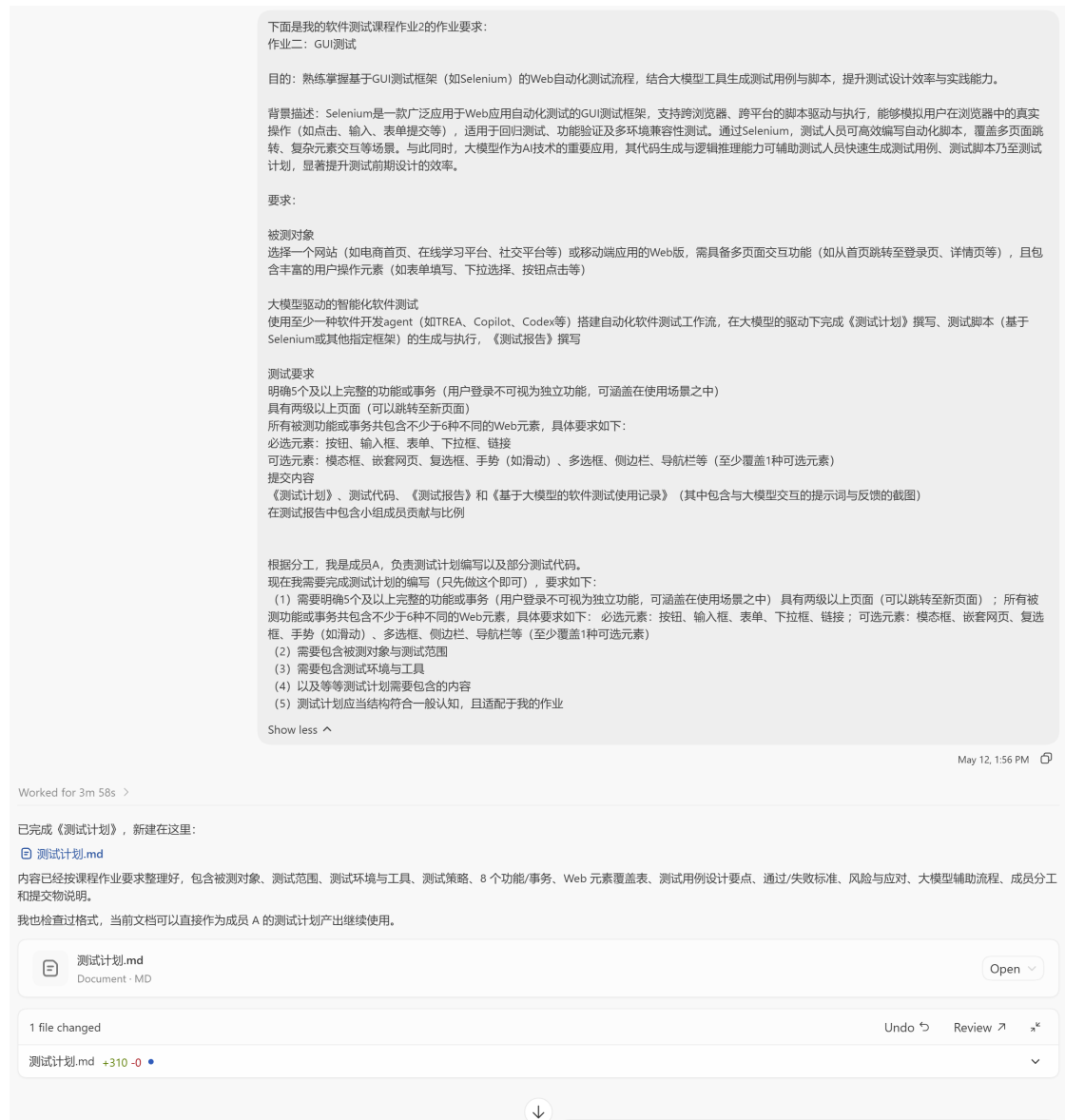


图 1：向大模型说明作业要求和个人分工，启动测试计划编写。



图 2：根据课程要求和报告结构，对测试计划内容进行精简和调整。



图 3：进一步讨论移动端 Web UI 测试范围、环境搭建方式和回退方案。

1.2 测试代码生成与执行

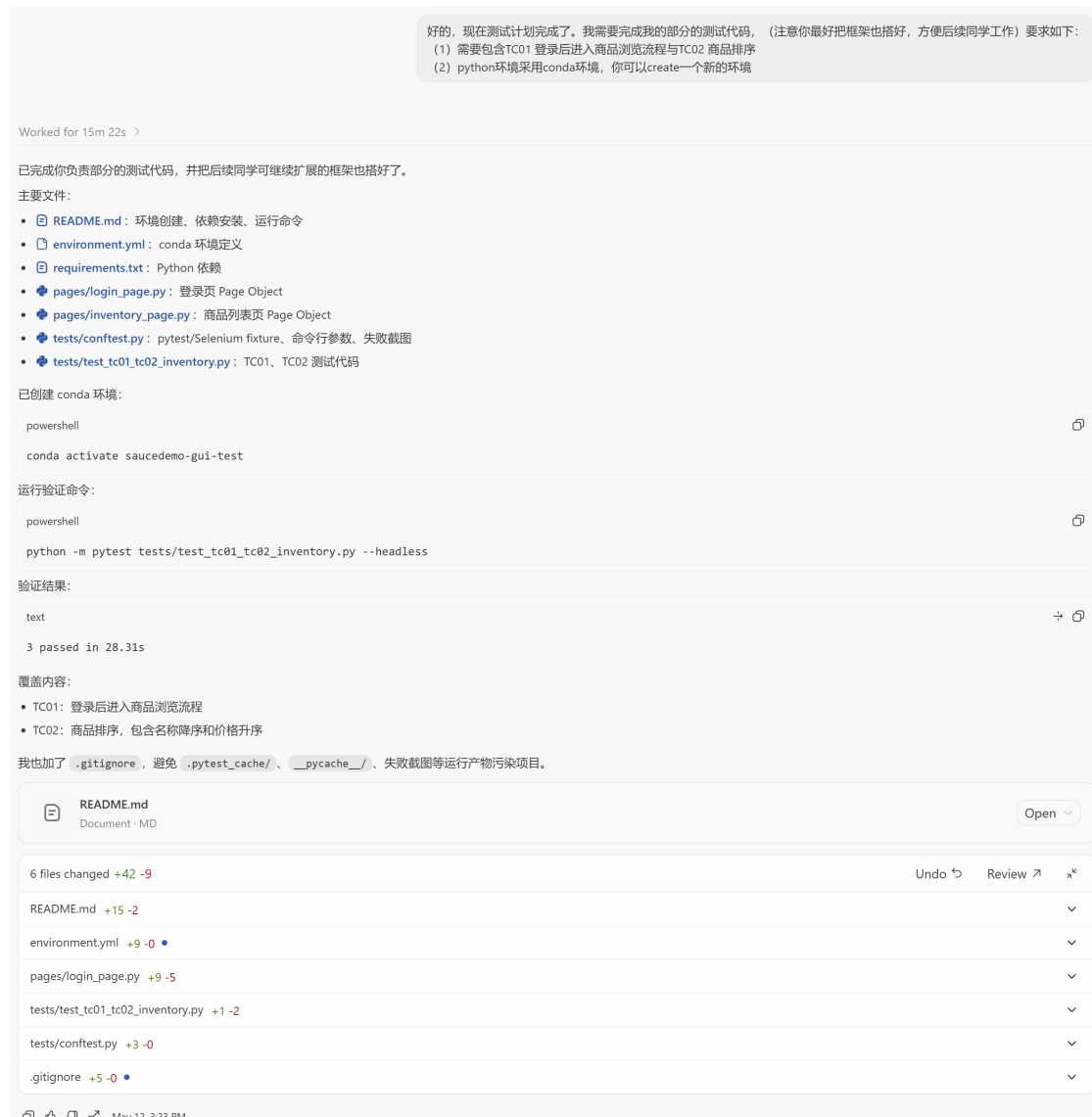


图 4：要求大模型搭建 Selenium 自动化测试框架并实现 TC01、TC02 测试脚本。

1.3 测试报告编写与修改



图 5：向大模型说明测试报告撰写要求，并生成报告初稿。

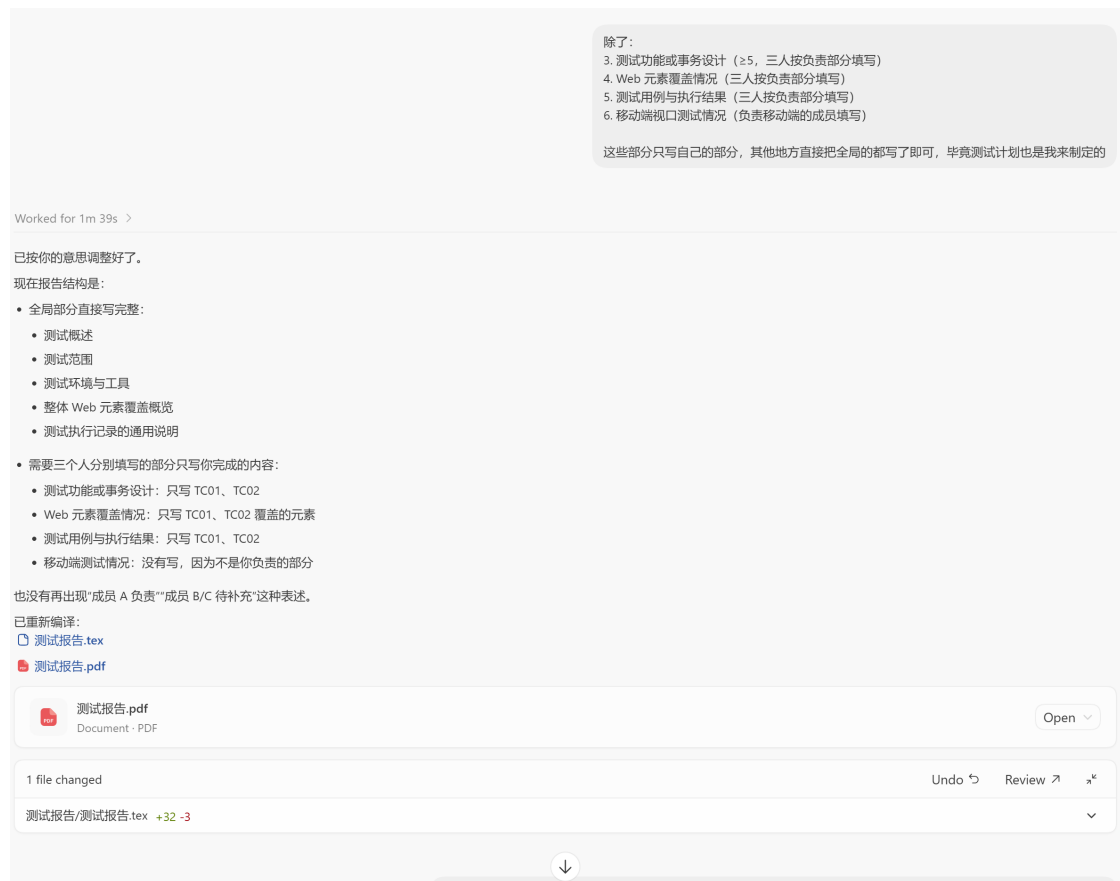


图 6：围绕报告表述方式、章节划分和成员内容边界进行修改。



图 7：继续优化测试报告中的元素覆盖表、执行结果截图和成员贡献部分。

2 成员 B 使用记录

2.1 测试代码生成与执行



图 8：向大模型说明成员 B 分工和测试计划要求，生成 TC03、TC04、TC05 商品详情与购物车测试代码。



图 9: 根据代码组织清晰度要求，将成员 B 测试拆分为 TC03、TC04、TC05 三个独立测试文件，并重新运行验证。

2.2 测试报告编写与修改



图 10: 要求大模型根据分工、测试代码和结果截图，在测试报告中补充成员 B 负责的测试用例与执行结果。



图 11：针对测试报告中 TC04 进入商品详情页方式和 TC05 数量表述进行修改，使成员 B 相关内容更清晰。

3 成员 C 使用记录

3.1 Web 端测试代码生成与执行

移动端测试是不是需要额外配置环境？你先完成普通的TC06

我先看一下项目结构和测试用例命名，确认 TC06 在现有测试体系里该落在哪里；移动端环境的问题我也会顺手从配置里判断一下。

我找到了普通 Web UI 用例已经做到 TC05，仓库里还有一份 Appium 移动端环境参考，所以移动端大概率是另一套环境；我先按你说的补普通版 TC06。Git 状态被 safe.directory 拦住了，我暂时不改全局配置，继续用文件内容推进。

TC06 的范围已经很明确：购物车点 Checkout，填写 first/last/postal code，进入 overview，再 Finish 到 complete 页。我会补一个 CheckoutPage 页面对象和对应 pytest 用例，顺便把 marker/README 更新到 TC06。

接下来开始改文件：CartPage 增加进入结算动作，新建 CheckoutPage 封装结算页/确认页/完成页，再加 test_tc06_checkout.py。这会保持现有 Page Object 风格，测试里只写业务断言。

已创建 2 个文件已编辑 3 个文件

代码已经补上了。现在跑 TC06 单测做验证；如果卡在访问 SauceDemo 或浏览器驱动网络问题，我会按环境限制处理。

验证先停在依赖层：当前 Python 环境没有 selenium。我先按项目的 requirements.txt 安装依赖，然后再跑 TC06。

已编辑 5 个文件
查看更改 ↗

撤销 ↶ 审核

README.md	+17 -9	▼
pages/cart_page.py	+11 -8	▼
pages/checkout_page.py	+123 -0	▼

再显示 2 个文件 ▼

14:14

(saucedemo-gui-test) PS E:\homework\software-test-hw2> pip show selenium
Name: selenium
Version: 4.44.0
你需要激活环境

已处理 2m 6s >

已完成普通版 TC06，并用你的 saucedemo-gui-test 环境验证通过。

改动：

- 新增 pages/checkout_page.py，封装结算信息页、订单确认页、订单完成页。
- 更新 pages/cart_page.py，增加 checkout()。
- 新增 tests/test_tc06_checkout.py，覆盖添加商品、进入结算、填写表单、确认订单、完成订单。
- 更新 pytest.ini 和 README.md 的 TC06 说明。

验证结果：

```
powershell  
conda run -n saucedemo-gui-test python -m pytest tests\test_tc06_checkout.py --headless
```

结果：1 passed。只有一个 .pytest_cache 写入权限 warning，不影响测试结果。

移动端测试：如果按你们计划里的 Selenium Chrome mobile emulation/窗口尺寸模拟，一般不需要额外搭 Appium 环境；如果要跑真实手机或 Android 模拟器，那才需要 Appium、Android SDK、设备/模拟器等额外配置。

README.md
文档 · MD

打开 ▼

↓

图 12: 向大模型说明先完成普通 Web 端 TC06 的需求，生成结算与订单完成相关的测试代码和页面对象。

```
(saucedemo-gui-test) PS E:\homework\software-test-hw2> python -m pytest
tests/test_tc06_checkout.py
===== test session starts =====
platform win32 -- Python 3.11.15, pytest-9.0.3, pluggy-1.6.0 -- D:\Anaconda\envs\saucedemo-
gui-test\python.exe
cachedir: .pytest_cache
rootdir: E:\homework\software-test-hw2
configfile: pytest.ini
collected 1 item

tests/test_tc06_checkout.py::test_tc06_checkout_completes_order FAILED [100%]

===== FAILURES =====
_____ test_tc06_checkout_completes_order _____

logged_in_inventory = <pages.inventory_page.InventoryPage object at
0x000002A17f6D3850>
...
显示更多
```

已处理 55s >

原因找到了：你非 headless 跑的时候，页面滚动到底部后 `Continue` 按钮虽然可见，但 Selenium 的普通 `.click()` 没有稳定触发，所以一直停在 `checkout-step-one.html`，最后等 `/checkout-step-two.html` 超时。

我已修复 [pages/checkout_page.py](#)：点击 `Continue` 和 `Finish` 前先把按钮滚动到视口中心，再点击。

我用和你一样的非 headless 命令验证过了：

```
powershell
conda run -n saucedemo-gui-test python -m pytest tests/test_tc06_checkout.py
```

结果： `1 passed`。

你现在直接再跑：

```
powershell
python -m pytest tests/test_tc06_checkout.py
```

应该就能过了，那个 `pytest_cache` 权限 warning 不影响测试通过。

已编辑 checkout_page.py
+9 -2

撤销 审核

详情

headless 是什么？你刚才解决的问题是什么？现在是否和其它 test 一致？

headless 是“无界面模式”。也就是 Chrome 在后台运行，不弹出真实浏览器窗口，适合自动化测试、CI、批量跑用例。不加 `--headless` 时就是“有界面模式”，会打开浏览器窗口，你能看到页面跳转和点击过程。

我刚才解决的问题是：TC06 在有界面模式下，页面滚动到底部后，`Continue` 按钮虽然显示出来了，但 Selenium 的普通点击没有稳定触发，导致页面没有从：

```
test
/checkout-step-one.html
```

跳到：

```
test
/checkout-step-two.html
```

所以测试一直等确认页，最后超时失败。

我做的修复是：在点击 `Continue` 和 `Finish` 之前，先把按钮滚动到浏览器视口中心，再执行点击。这样更接近真实用户看到按钮后点击的行为，也更稳定。

现在和其它 test 是一致的：

- 都是普通 Selenium + pytest 测试；
- 都使用 Page Object 写法；
- 都默认可以直接用 `python -m pytest ...` 跑；
- 不需要强制加 `--headless`；
- `--headless` 只是可选运行参数。

你现在跑这个就可以：

```
powershell
python -m pytest tests/test_tc06_checkout.py
```

TC06 已经按普通桌面端 Web UI 测试完成。

图 13：根据本地运行结果，与大模型一起定位 TC06 在非 headless 模式下点击 Continue 不稳定的问题，并通过滚动到按钮后点击的方式修复，最终运行通过。

3.2 移动端测试计划与环境确认



图 14: 讨论移动端测试是否需要额外环境,并确定采用 Appium、Android 模拟器和 Android Chrome 进行真实移动端 Web UI 测试。



图 15: 在已安装 Android Studio、模拟器、adb 和 Appium 的基础上, 向大模型确认后续步骤, 并生成 Appium 冒烟测试和移动端测试代码。

3.3 移动端测试错误定位与修复



图 16: 运行 Appium 冒烟测试时遇到 Chromedriver 与 Android Chrome 版本不匹配问题，和大模型一起分析原因并调整 Appium 启动方式。


```
> raise exception_class(msg=message, stacktrace=format_stacktrace(stacktrace))
E selenium.common.exceptions.InvalidArgumentException: Message: invalid argument:
invalid locator
E (Session info: chrome=124.0.6367.219)
E Stacktrace:
E   GetHandleVerifier [0x00007FF78F3F1522+60802]
E   (No symbol) [0x00007FF78F36AC22]
E   (No symbol) [0x00007FF78F227CE4]
E   (No symbol) [0x00007FF78F2763E8]
E   (No symbol) [0x00007FF78F276E1C]
E   (No symbol) [0x00007FF78F2BCE37]
E   (No symbol) [0x00007FF78F29ABBF]
E   (No symbol) [0x00007FF78F2BA224]
E   (No symbol) [0x00007FF78F29A923]
E   (No symbol) [0x00007FF78F268FEC]
E   (No symbol) [0x00007FF78F269C21]
E   GetHandleVerifier [0x00007FF78F6F41BD+3217949]
E   GetHandleVerifier [0x00007FF78F736157+3488183]
E   GetHandleVerifier [0x00007FF78F72F0DF+3459391]
E   GetHandleVerifier [0x00007FF78F4A88E6+823622]...
显示更多 v
```

17:03

已处理 1m 27s >

这是定位器兼容问题，不是 Appium 环境问题了。你已经过了最难的 Chromedriver 匹配那关。

原因：Appium + Android Chrome 这条链路里，`By.ID` / `By.CLASS_NAME` 有时会被传成移动端 Chromedriver 不接受的 locator strategy，于是报：

```
text
invalid argument: invalid locator
```

我已把 Appium 测试里的定位方式统一改成 CSS selector，例如：

```
python
(By.CSS_SELECTOR, "#login-button")
(By.CSS_SELECTOR, ".title")
```

并验证通过：

```
text
tests/test_appium_smoke.py::test_open_saucedemo_in_android_chrome PASSED
tests/test_tc07_mobile_appium.py::test_tc07_mobile_appium_login_scroll_and_add_to_cart PASSED
```

你现在可以直接运行：

```
powershell
python -m pytest tests/test_appium_smoke.py
python -m pytest tests/test_tc07_mobile_appium.py
```

注意 Appium 仍然要保持用这个命令启动：

```
powershell
appium --allow-insecure chromedriver_autodownload
```

已编辑 2 个文件
+23 -11

撤销 审核

tests/test_appium_smoke.py	+1 -1 v
tests/test_tc07_mobile_appium.py	+22 -10 v

图 17：继续处理移动端测试中的 locator 兼容问题和点击稳定性问题，将定位方式调整为 CSS selector，并优化移动端页面点击逻辑。



图 18：根据测试计划编号重新梳理 TC07 与移动端测试的关系，将侧边栏导航菜单保留为 TC07，并将 Appium 移动端综合测试调整为 TC08。

3.4 测试报告编写与修改



图 19: 要求大模型根据成员 C 负责的 TC06、TC07 和 TC08 测试内容，在测试报告中补充对应测试步骤、检查点、执行结果和截图引用。



图 20: 继续与大模型讨论测试报告结构和表述方式，对桌面端与移动端测试环境、问题与结论、UI 截图展示等内容进行调整。